

Trabajo Práctico Integrador

Informe técnico

Sistema de gestión de propiedades

Programación II

Grupo 135

Monasterio, María Sofía

Elección del dominio y justificación

Dominio seleccionado

Se eligió el dominio **Propiedad -> Escritura Notarial**, modelando la relación entre propiedades inmobiliarias y sus documentos notariales asociados.

Justificación

Esta elección se fundamenta en los siguientes aspectos:

- Relevancia del negocio: Este dominio refleja una situación real conocida.
- Adecuación a los requisitos técnicos: Permite modelar una relación 1:1 clara: una propiedad puede tener una escritura pero no todas las propiedades la tienen.

Diseño

Relación 1:1 unidireccional

Se implementó la relación 1+1 unidireccional donde solo **Propiedad** hace referencia a **EscrituraNotarial**.

Justificación:

- Simplicidad: La unidireccionalidad reduce la complejidad del modelo y evita referencias circulares.
- Responsabilidad única: La Propiedad es quien necesita conocer su documentación, no al revés.

Implementación:

```
public class Propiedad {  
    private EscrituraNotarial escrituraNotarial; // Referencia unidireccional  
    // ... otros atributos  
}
```

FK única vs PK compartida

Se definió utilizar una clave foránea única en lugar de una clave primaria compartida.

```
CREATE TABLE propiedad (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    escritura_id BIGINT UNIQUE,  
    FOREIGN KEY (escritura_id) REFERENCES escritura_notarial(id)  
);
```

Justificación:

- Flexibilidad: Permite que una propiedad exista sin escritura asociada.
- Independencia: Las escrituras pueden crearse independientemente antes de asociarse a una propiedad.

- Mantenibilidad: Es más fácil agregar o quitar la relación sin afectar la estructura de las tablas.
- Claridad: La relación es explícita y fácil de entender en el esquema de base de datos.
- Implementación de la restricción UNIQUE: La restricción `UNIQUE KEY uk\_escritura\_id (escritura\_id)` garantiza que cada escritura solo puede estar asociada a una propiedad y se cumpla la cardinalidad 1→1 de la relación.

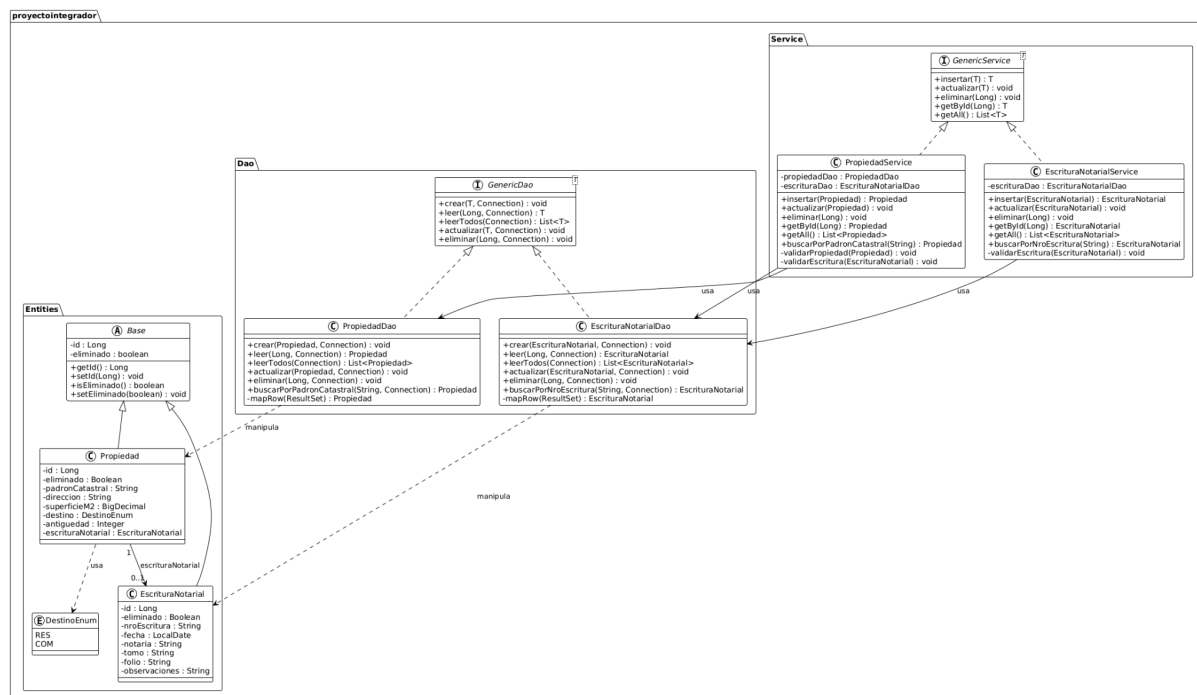
UML

Paquetes principales:

- Entities: Clases de Dominio (Base, Propiedad, EscrituraNotarial, DestinoEnum)
- Dao: Capa de acceso de datos (GenericDao, PropiedadDao, EscrituraNotarialDao)
- Service: Capa de servicios (GenericService, PropiedadService, EscrituraNotarialService)

Relaciones:

- Propiedad -> EscrituraNotarial (1:1 unidireccional)
- PropiedadDao implementa GenericDao<Propiedad>
- PropiedadService implementa GenericService<Propiedad>
- PropiedadService usa PropiedadDao y EscrituraNotarialDao



Arquitectura por capas

El sistema separa responsabilidades según el principio de responsabilidad única, siguiendo una arquitectura en capas.

Capa de entidades

Se encarga de modelar las entidades del dominio, además de contener la lógica de datos con los getters, setters y constructores. Define la estructura de los objetos del negocio.

Capa de acceso de datos

Es la capa DAO. Su función es abstraer el acceso a la base de datos, implementando las operaciones CRUD usando JDBC. También mapea las consultas SQL a objetos Java.

Utiliza preparedStatements en todas las consultas para prevenir inyecciones de SQL.

Los métodos aceptan conexiones externas a través del parámetro Connection.

El método eliminar usa la baja lógica, que actualiza el campo eliminado con un booleano en lugar de borrar físicamente en la base de datos.

Capa de servicios

Es la encargada de implementar las reglas del negocio.

Orquesta transacciones, coordina operaciones entre los distintos DAOs y valida los datos antes de persistirlos en la base de datos.

Capa de configuración

Gestiona la conexión con la base de datos.

Es donde se cargan los archivos de configuración de la base y se registra el driver JDBC.

Capa de presentación

Acá se encuentra el menú de consola que se presenta al usuario y a través del cual se capturan los datos de entrada.

Es un menú interactivo con opciones numeradas, donde se validan las diferentes entradas y se muestran las diferentes salidas, incluyendo los mensajes de error.

Persistencia: Estructura de Base de Datos y transacciones

Estructura de la base de datos

```
**Tabla: `escritura_notarial`**
CREATE TABLE escritura_notarial (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  eliminado BOOLEAN NOT NULL DEFAULT FALSE,
  nro_escritura VARCHAR(50) NOT NULL,
  fecha DATE NOT NULL,
  notaria VARCHAR(100) NOT NULL,
  tomo VARCHAR(20),
  folio VARCHAR(20),
  observaciones TEXT,
  INDEX idx_nro_escritura (nro_escritura),
  INDEX idx_eliminado (eliminado)
);
```

```
**Tabla: `propiedad`**
CREATE TABLE propiedad (
```

```

id BIGINT AUTO_INCREMENT PRIMARY KEY,
eliminado BOOLEAN NOT NULL DEFAULT FALSE,
padron_catastral VARCHAR(50) NOT NULL UNIQUE,
direccion VARCHAR(200) NOT NULL,
superficie_m2 DECIMAL(10,2) NOT NULL,
destino ENUM('RES', 'COM') NOT NULL,
antiguedad INT NOT NULL,
escritura_id BIGINT,
INDEX idx_padron_catastral (padron_catastral),
INDEX idx_eliminado (eliminado),
INDEX idx_escritura_id (escritura_id),
UNIQUE KEY uk_escritura_id (escritura_id),
FOREIGN KEY (escritura_id)
    REFERENCES escritura_notarial(id)
    ON DELETE SET NULL
    ON UPDATE CASCADE
);

```

Orden de las operaciones

Crear propiedad con escritura

1. Abrir conexión y desactivar auto-commit.
2. Validar datos de escritura.
3. Crear escritura notarial (obtener ID generado).
4. Validar datos de propiedad.
5. Asociar ID de escritura a propiedad.
6. Crear propiedad.
7. Commit si todo es exitoso, rollback si hay error.

Actualizar propiedad

1. Abrir conexión y desactivar auto-commit.
2. Verificar que la propiedad existe.
3. Si se actualiza la escritura asociada:
 - Si escritura tiene ID: actualizar escritura existente.
 - Si escritura no tiene ID: crear nueva escritura.
4. Actualizar propiedad.
5. Commit o rollback según resultado.

Eliminar

1. Abrir conexión y desactivar auto-commit.
2. Verificar que el registro existe.
3. Actualizar campo `eliminado = true`.

4. Commit.

Transacciones: Commit y Rollback

Transacciones: Todas las operaciones de escritura están en la capa de servicio.

```
Connection conn = null;
try {
    conn = DatabaseConnection.getConnection();
    conn.setAutoCommit(false); // Iniciar transacción

    // Operaciones de negocio
    escrituraDao.crear(escritura, conn);
    propiedadDao.crear(propiedad, conn);

    conn.commit(); // Confirmar si todo es exitoso
} catch (Exception e) {
    if (conn != null) {
        conn.rollback(); // Revertir en caso de error
    }
    throw new Exception("Error: " + e.getMessage(), e);
} finally {
    if (conn != null) {
        conn.setAutoCommit(true); // Restaurar auto-commit
        conn.close();
    }
}
```

Casos de uso de transacciones:

- Crear propiedad con escritura: Si falla la creación de la escritura, no se crea la propiedad.
- Actualizar propiedad y escritura: Si falla la actualización de la escritura, se revierte todo el proceso.
- Eliminar.

Validaciones y reglas de negocio

Propiedad

Campos obligatorios

- Padrón Catastral: No puede estar vacío y debe ser único en la base de datos.
- Dirección: No ser nula.

- Superficie: Debe ser mayor a 0.
- Destino: De ser RES o COM.
- Antigüedad: Debe ser mayor o igual a 0.

Escritura Notarial

Campos obligatorios:

- Número de escritura: No puede ser nulo.
- Fecha: No puede ser nula.
- Notaria: No puede ser nula.

Campos opcionales:

- Tomo, Folio y Observaciones.

Reglas de negocio

1. Relación 1 -> 1
 - a. Una propiedad puede tener máximo una escritura asociada
 - b. Una escritura solo puede estar asociada a una propiedad
2. Baja Lógica
 - a. Los registros eliminados no se borran físicamente
 - b. Se marca eliminado = true
3. Unicidad de Padrón Catastral
 - a. Cada padrón debe ser único en la base

Manejo de errores

Las validaciones de los errores se realizan en tres niveles:

1. Interfaz de usuario: se validan los formatos de números y fechas que ingresa el usuario.
2. Capa de servicio: Válida las reglas de negocio y los campos obligatorios.
3. Base de datos: Válida restricciones de integridad

Pruebas

Pruebas funcionales

CRUD Propiedades

- Crear propiedad sin escritura
- Crear propiedad con escritura nueva
- Crear propiedad con escritura existente
- Buscar por ID
- Buscar por padrón catastral
- Listar todas las propiedades
- Actualizar propiedad
- Eliminar propiedad (baja lógica)

CRUD Escrituras

- Crear escritura

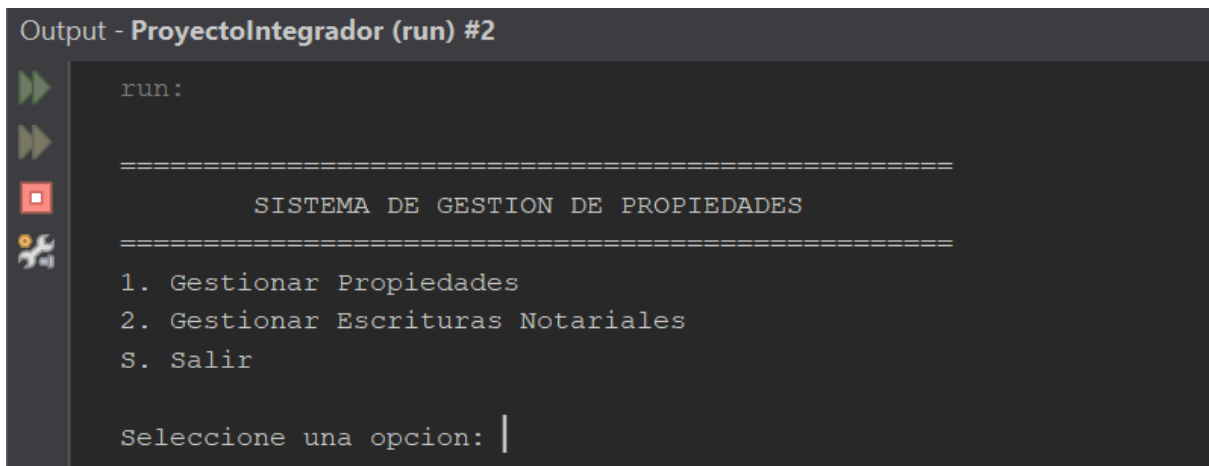
- Buscar por ID
- Buscar por número
- Listar todas las escrituras
- Actualizar escritura
- Eliminar escritura (baja lógica)

Pruebas de validación

- Crear propiedad sin padrón catastral: Error esperado
- Crear propiedad con superficie ≤ 0 : Error esperado
- Crear propiedad con destino inválido: Error esperado
- Crear escritura sin número: Error esperado
- Crear escritura sin fecha: Error esperado
- Crear escritura sin notaría: Error esperado

Capturas del sistema

Menú Principal



```
Output - ProyectoIntegrador (run) #2

run:

=====
                SISTEMA DE GESTION DE PROPIEDADES
=====

1. Gestionar Propiedades
2. Gestionar Escrituras Notariales
S. Salir

Seleccione una opcion: |
```


Menú Propiedades

```
Output - ProyectoIntegrador (run)
S. Salir
Seleccione una opcion: 1
-----
MENU DE PROPIEDADES
-----
1. Crear Propiedad
2. Buscar Propiedad por ID
3. Buscar Propiedad por Padron Catastral
4. Listar Todas las Propiedades
5. Actualizar Propiedad
6. Eliminar Propiedad
V. Volver al menu principal
Seleccione una opcion:
```

Menú Escrituras

```
Output - ProyectoIntegrador (run)
1. Gestionar Propiedades
2. Gestionar Escrituras Notariales
S. Salir
Seleccione una opcion: 2
-----
MENU DE ESCRITURAS NOTARIALES
-----
1. Crear Escritura Notarial
2. Buscar Escritura por ID
3. Buscar Escritura por Numero
4. Listar Todas las Escrituras
5. Actualizar Escritura
6. Eliminar Escritura
V. Volver al menu principal
Seleccione una opcion:
```

Consultas SQL

Ver todas las propiedades con sus escrituras

```
SELECT
    p.id,
    p.padron_catastral,
    p.direccion,
    p.superficie_m2,
    p.destino,
    p.antiguedad,
    e.nro_escritura,
    e.fecha,
    e.notaria
FROM propiedad p
LEFT JOIN escritura_notarial e ON p.escritura_id = e.id
WHERE p.eliminado = FALSE;
```

Buscar propiedades sin escrituras

```
SELECT *
FROM propiedad
WHERE eliminado = FALSE
AND escritura_id IS NULL;
```

Propiedades eliminadas

```
SELECT COUNT(*) as eliminadas
FROM propiedad
WHERE eliminado = TRUE;
```

Conclusiones y mejoras futuras

Se logró crear un sistema CRUD funcional completo para las dos entidades planteadas.

Se estableció la relación 1:1 solicitada.

Las transacciones a la base de datos funcionan con commit y rollback.

Se trabajó la arquitectura en capas vista durante el cursado.

Algunos aprendizajes:

- La separación en capas, (DAO, service, entities) para facilitar el mantenimiento y la legibilidad del código.
- Aplicación de validaciones en múltiples capas mejorando la robustez del sistema.
- Implementación del patrón DAO para evitar código duplicado.

Mejoras futuras:

- Implementación de una interfaz gráfica.
- Permitir búsquedas por varios criterios o filtros.
- Historial de auditoría de los cambios realizados

Fuentes y herramientas utilizadas

Tecnologías y Frameworks

- Java 21: Lenguaje de programación principal.
- MySQL 8.0: Sistema de gestión de bases de datos relacional.
- MySQL Connector/J 8.0+: Driver JDBC para conexión a MySQL.
- JDBC: API para acceso a bases de datos desde Java.

Herramientas de Desarrollo

- NetBeans IDE: Entorno de desarrollo integrado.
- DBeaver: Herramienta de gestión de bases de datos.
- Git: Control de versiones.

Herramientas de Documentación

- PlantUML: Para generación del diagrama UML.
- draw.io: Para diagramas UML.

Recursos y Documentación

Documentación oficial:

- Oracle Java Documentation: <https://docs.oracle.com/javase/>
- MySQL Documentation: <https://dev.mysql.com/doc/>
- JDBC Tutorial: <https://docs.oracle.com/javase/tutorial/jdbc/>

Uso de Inteligencia Artificial

Herramienta utilizada: Cursor AI / ChatGPT.

Asistencia proporcionada:

- Resolución de problemas de configuración (db.properties, driver JDBC).
- Creación de documentación (README).
- Generación de diagramas UML en formato PlantUML.
- Sugerencias de mejoras y optimizaciones.
- Resolución de errores de compilación y runtime.

Nota sobre el uso de IA:

La IA fue utilizada como herramienta de asistencia en el desarrollo, similar a como se usaría Stack Overflow o documentación. Todo el código fue revisado, probado y comprendido antes de su implementación. Las decisiones de diseño y arquitectura fueron tomadas considerando los requisitos del trabajo práctico y las mejores prácticas de desarrollo.